

Classification Assignment

Problem Statement or Requirement:

A requirement from the Hospital, Management asked us to create a predictive model which will predict the chronic kidney disease (CKD) based on the several parameters. The Client has provided the dataset of the same.

1.) Identify your problem statement

Here We need to find the chronic kidney disease is present or not. So the output is categorical data.

Machine Learning-> Supervised learning -> Classification

2) Tell basic info about the dataset (Total number of rows, columns)

Columns = 25

Rows= 399

Independent variable = 24 variables

Dependent variable = 1

3) Mention the pre-processing method if you're doing any (like converting string to number – nominal data)

Ordinal Method – used to convert for categorical data rbc, pc...etc.,

4) Develop a good model with good evaluation metric. You can use any machine learning algorithm; you can create many models. Finally, you must come up with final model.

1) Decision Tree:

Accuracy: 0.99

```
[7]: from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, Y_pred)
cm
```

```
[7]: array([[45,  0],
        [ 1, 74]])
```

```
[8]: from sklearn.metrics import classification_report
clf_report = classification_report(Y_test, Y_pred)
print(clf_report)
```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	45
1	1.00	0.99	0.99	75
accuracy			0.99	120
macro avg	0.99	0.99	0.99	120
weighted avg	0.99	0.99	0.99	120

Decision Tree with preprocessing:

Accuracy: 0.99

```
[6]: from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, Y_pred)
cm
```

```
[6]: array([[45,  0],
        [ 1, 74]])
```

```
[7]: from sklearn.metrics import classification_report
clf_report = classification_report(Y_test, Y_pred)
print(clf_report)
```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	45
1	1.00	0.99	0.99	75
accuracy			0.99	120
macro avg	0.99	0.99	0.99	120
weighted avg	0.99	0.99	0.99	120

2. Random Forest

Accuracy = 0.98

```
[4]: from sklearn.metrics import confusion_matrix
cm= confusion_matrix(Y_test, Y_pred)
cm
```

```
[4]: array([[44,  1],
          [ 1, 74]])
```

```
[5]: from sklearn.metrics import classification_report
clf_report = classification_report(Y_test, Y_pred)
print(clf_report)
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	45
1	0.99	0.99	0.99	75
accuracy			0.98	120
macro avg	0.98	0.98	0.98	120
weighted avg	0.98	0.98	0.98	120

```
[ ]:
```

Random Forest with Preprocessing:

Accuracy = 0.98

```
from sklearn.metrics import confusion_matrix
cm= confusion_matrix(Y_test, Y_pred)
cm
```

```
array([[44,  1],
       [ 1, 74]])
```

```
from sklearn.metrics import classification_report
clf_report = classification_report(Y_test, Y_pred)
print(clf_report)
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	45
1	0.99	0.99	0.99	75
accuracy			0.98	120
macro avg	0.98	0.98	0.98	120
weighted avg	0.98	0.98	0.98	120

3. SVM

Accuracy = 0.98

```
[3]: from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(Y_test, Y_pred)  
cm
```

```
[3]: array([[44,  1],  
         [ 1, 74]])
```

```
[4]: grid.best_estimator_
```

```
[4]: SVC  
Parameters
```

```
[5]: from sklearn.metrics import classification_report  
clr = classification_report(Y_test, Y_pred)  
print(clr)
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	45
1	0.99	0.99	0.99	75
accuracy			0.98	120
macro avg	0.98	0.98	0.98	120
weighted avg	0.98	0.98	0.98	120

SVM with Preprocessing:

Accuracy = 0.97

```
[3]: from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(Y_test, Y_pred)  
cm
```

```
[3]: array([[45,  0],  
         [ 3, 72]])
```

```
[5]: from sklearn.metrics import classification_report  
clr = classification_report(Y_test, Y_pred)  
print(clr)
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	45
1	1.00	0.96	0.98	75
accuracy			0.97	120
macro avg	0.97	0.98	0.97	120
weighted avg	0.98	0.97	0.98	120

4) Logistic Regression:

Accuracy = 0.97

```
[3]: Y_pre= grid.predict(X_test)

      from sklearn.metrics import confusion_matrix
      cm = confusion_matrix(Y_test, Y_pre)

[4]: print(cm)
      from sklearn.metrics import classification_report

      print(classification_report(Y_test, Y_pre))
```

```
[[44  1]
 [ 2 73]]
```

		precision	recall	f1-score	support
	0	0.96	0.98	0.97	45
	1	0.99	0.97	0.98	75
	accuracy			0.97	120
	macro avg	0.97	0.98	0.97	120
	weighted avg	0.98	0.97	0.98	120

Logistic Regression with Preprocessing:

Accuracy = 0.99

```
print(cm)
from sklearn.metrics import classification_report

print(classification_report(Y_test, Y_pre))
```

```
[[45  0]
 [ 1 74]]
```

		precision	recall	f1-score	support
	0	0.98	1.00	0.99	45
	1	1.00	0.99	0.99	75
	accuracy			0.99	120
	macro avg	0.99	0.99	0.99	120
	weighted avg	0.99	0.99	0.99	120

5) KNN

Accuracy = 0.80

```
[3]: Y_pred= grid.predict(X_test)

from sklearn.metrics import confusion_matrix
cmx= confusion_matrix(Y_test, Y_pred)
print(cmx)

[[33  3]
 [17 47]]
```

```
[4]: from sklearn.metrics import classification_report
cr = classification_report(Y_test, Y_pred)
print(cr)
```

	precision	recall	f1-score	support
0	0.66	0.92	0.77	36
1	0.94	0.73	0.82	64
accuracy			0.80	100
macro avg	0.80	0.83	0.80	100
weighted avg	0.84	0.80	0.80	100

KNN with Preprocessing

Accuracy = 0.97

```
from sklearn.metrics import classification_report
cr = classification_report(Y_test, Y_pred)
print(cr)
```

	precision	recall	f1-score	support
0	0.92	1.00	0.96	36
1	1.00	0.95	0.98	64
accuracy			0.97	100
macro avg	0.96	0.98	0.97	100
weighted avg	0.97	0.97	0.97	100

6. Naive Bayes:

Accuracy = 0.97

```
Y_pred= grid.predict(X_test)
from sklearn.metrics import confusion_matrix
cm= confusion_matrix(Y_test, Y_pred)
print(cm)
```

```
[[45  0]
 [ 3 72]]
```

```
from sklearn.metrics import classification_report
clr= classification_report(Y_test,Y_pred)
print(clr)
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	45
1	1.00	0.96	0.98	75
accuracy			0.97	120
macro avg	0.97	0.98	0.97	120
weighted avg	0.98	0.97	0.98	120

Naive Bayes with preprocessing:

Accuracy = 0.97

```
Y_pred= grid.predict(X_test)
from sklearn.metrics import confusion_matrix
cm= confusion_matrix(Y_test, Y_pred)
print(cm)
```

```
[[45  0]
 [ 3 72]]
```

```
from sklearn.metrics import classification_report
clr= classification_report(Y_test,Y_pred)
print(clr)
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	45
1	1.00	0.96	0.98	75
accuracy			0.97	120
macro avg	0.97	0.98	0.97	120
weighted avg	0.98	0.97	0.98	120

7. XGBoost

Accuracy = 0.98

```
[3]: from sklearn.metrics import confusion_matrix
cm= confusion_matrix(Y_test, Y_pred)
print(cm)
```

```
[[44  1]
 [ 1 74]]
```

```
[4]: from sklearn.metrics import classification_report
clr= classification_report(Y_test,Y_pred)
print(clr)
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	45
1	0.99	0.99	0.99	75
accuracy			0.98	120
macro avg	0.98	0.98	0.98	120
weighted avg	0.98	0.98	0.98	120

.

XGBoost with Preprocessing:

Accuracy = 0.98

```
: from sklearn.metrics import confusion_matrix
cm= confusion_matrix(Y_test, Y_pred)
print(cm)
```

```
[[44  1]
 [ 1 74]]
```

```
: from sklearn.metrics import classification_report
clr= classification_report(Y_test,Y_pred)
print(clr)
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	45
1	0.99	0.99	0.99	75
accuracy			0.98	120
macro avg	0.98	0.98	0.98	120
weighted avg	0.98	0.98	0.98	120

Overall, **Decision Tree & Logistic Regression with Preprocessing:**

is best model: Because Accuracy: 0.99